



DIEGO ALMEIDA DA SILVA

PROJETO CLÍNICA VETERINÁRIA

Fundação de Apoio à Escola Técnica
Estado do Rio de Janeiro

DIEGO ALMEIDA DA SILVA

PROJETO CLÍNICA VETERINÁRIA

Projeto apresentado ao Curso Técnico em Informática como requisito final para a obtenção do título de Técnico em Informática, sob a orientação do Prof. Fabiano.

Documentos De Requisitos

Sistema para Controle de Clínica Veterinária

versão 1.0

11/06/2026

ETAPA 1- Modelagem do Sistema

PARTE 1 – passo 1: Levantamento de Requisitos

1.1 - Objetivo

Este documento destina-se aos clientes, desenvolvedores e responsáveis envolvidos no desenvolvimento de um sistema de software para controle de uma clínica veterinária. O seu objetivo é apresentar a descrição das funcionalidades e serviços que o sistema deve oferecer, bem como suas restrições de funcionamento e requisitos de qualidade. O sistema foi desenvolvido para auxiliar no gerenciamento das atividades da clínica veterinária, permitindo o controle de animais, tutores, veterinários, medicamentos e atendimentos realizados. Além disso, o software possibilita consultas de informações importantes, facilitando a organização e o acompanhamento dos serviços prestados pela clínica. Este documento apresenta detalhadamente os requisitos funcionais, requisitos não funcionais e regras de negócio necessários para o desenvolvimento do sistema, servindo como apoio para as etapas de análise, modelagem, implementação, testes e validação do software.

1.2 - Escopo

O sistema em questão será desenvolvido para realizar o controle das principais atividades de uma clínica veterinária. O software deverá permitir o cadastro e gerenciamento de animais, tutores, veterinários, medicamentos e atendimentos realizados na clínica. O sistema também será responsável por registrar consultas veterinárias, incluindo diagnósticos, medicamentos utilizados, tipo de atendimento e valor total da consulta. Além disso, deverá possibilitar o controle do estoque de medicamentos, auxiliando no monitoramento das medicações disponíveis na clínica. Como forma de auxiliar o trabalho dos funcionários e administradores, o sistema deverá disponibilizar consultas de animais atendidos por veterinário, listagem de tutores cadastrados, veterinários por turno e número de atendimentos realizados por ano. Também está no escopo do projeto o desenvolvimento de interfaces gráficas intuitivas, integração com banco de dados MySQL e funcionalidades básicas de manipulação de dados, como cadastro, alteração, exclusão e listagem de informações do sistema.

1.3. Convenções, Termos e Abreviações

1.3.1. Especificação dos requisitos

Esta seção descreve os requisitos funcionais, não funcionais e as regras de negócio necessárias para o funcionamento do sistema proposto. A especificação de cada um desses tipos de requisitos é efetuada por meio das convenções apresentadas a seguir.

- RF: Os requisitos funcionais são iniciados com a sigla RF, seguido de um identificador numérico único. Por exemplo, RF001, RF002 etc.
- RNF: Os requisitos não funcionais são iniciados com a sigla RNF, seguido de um identificador numérico único. Por exemplo, RNF001, RNF002 etc.
- RN: As regras de negócio são iniciadas com a sigla RN, seguido de um identificador numérico único. Por exemplo, RN001, RN002 etc.

1.3.2. Prioridades dos requisitos

Para determinar a prioridade dos requisitos, são adotadas as denominações Essencial, importante e desejável. Abaixo temos a descrição de cada um desses termos.

Essencial: Requisito na qual, sem o mesmo, o sistema não pode entrar em funcionamento.

Este tipo de requisito tem o nível mais alto de prioridade e deve ser implementado obrigatoriamente.

Importante: Requisito na qual, sem o mesmo, o sistema pode entrar em funcionamento, mas de forma não satisfatória.

Requisitos desse tipo agregam valor ao sistema, mas não comprometem o seu funcionamento.

Desejável: Requisito que não compromete as funcionalidades básicas do sistema. O sistema consegue funcionar de forma satisfatória sem eles.

Esse tipo de requisito pode ser implementado em versões posteriores do sistema.

1.4. Visão Geral

Esta introdução apresenta as informações necessárias para a correta utilização deste documento, explicando seus objetivos, o escopo do sistema e as convenções adotadas no texto. Além disso, contém uma lista de referências utilizadas para a sua elaboração. As demais seções estão organizadas da seguinte maneira:

A **Seção 2** apresenta uma descrição geral do sistema, incluindo as funcionalidades que o software deverá executar, os tipos de usuários envolvidos e as restrições e dependências do sistema.

A **Seção 3** apresenta o detalhamento dos requisitos funcionais, requisitos não funcionais e regras de negócio identificados durante o processo de levantamento de requisitos.

Descrição Geral e funcionalidades

2.1. Perspectiva do Produto

O software será desenvolvido como um sistema do tipo desktop e será utilizado nos computadores da clínica veterinária. O sistema funcionará apenas na rede interna da clínica, não podendo ser acessado via web ou por dispositivos móveis.

Os usuários irão interagir com o sistema por meio de uma interface gráfica compatível com o sistema operacional Windows, ambiente já utilizado pelos funcionários da clínica. Dessa forma, não será necessário instalar outros softwares além do próprio sistema e do sistema operacional citado.

O software proposto funcionará de forma independente, não sendo necessária a comunicação com outros sistemas externos para a execução de suas funcionalidades.

2.2. Funções do Produto

O sistema deve registrar os atendimentos realizados na clínica veterinária, identificando os animais, tutores e veterinários responsáveis. Além disso, deve calcular o valor total da consulta com base nos medicamentos utilizados.

O software deve permitir o cadastro de animais, tutores, veterinários e medicamentos, além do controle de estoque das medicações disponíveis.

O sistema também deve possibilitar o gerenciamento dos atendimentos, incluindo abertura, registro e finalização das consultas veterinárias.

Para auxiliar a administração da clínica, o sistema deverá fornecer consultas de animais atendidos por veterinário, veterinários por turno, tutores cadastrados, estoque de medicamentos e número de atendimentos por ano.

2.3. Características do Usuário

O sistema descrito neste documento será utilizado por três tipos de usuários:

- **Administrador:** Responsável pela administração do sistema. Possui conhecimento intermediário em informática e será responsável pelo cadastro de veterinários, medicamentos e consultas administrativas do sistema.
- **Veterinário:** Responsável pelos atendimentos realizados na clínica. Possui conhecimento básico em informática e será responsável pelo cadastro de animais, tutores e registro dos atendimentos veterinários.

2.4. Restrições

As restrições consideradas para o desenvolvimento do sistema são listadas abaixo:

- O software deverá funcionar no sistema operacional Windows;
- O sistema deverá consumir poucos recursos das máquinas da clínica, não ultrapassando 50% da memória disponível dos computadores;
- O código-fonte do sistema será desenvolvido utilizando a linguagem de programação Java;
- O armazenamento dos dados será realizado utilizando o Sistema de Gerenciamento de Banco de Dados MySQL;

- O sistema será desenvolvido como uma aplicação desktop, funcionando apenas nos computadores da clínica.

2.5. Suposições e Dependências

As seguintes suposições são consideradas durante o desenvolvimento do sistema:

- A clínica será responsável pela disponibilização dos computadores e da infraestrutura necessária para utilização do sistema;
- O cliente será responsável pela instalação e manutenção do sistema operacional Windows nos computadores utilizados;
- O cliente deverá disponibilizar os recursos de hardware e software necessários para a implantação do sistema;
- O banco de dados MySQL deverá estar instalado e configurado para o correto funcionamento do sistema.

Requisitos Específicos

3.1. Requisitos Funcionais

Identificador	RF 0001
Nome	Manter Cadastro de Tutores
Descrição	O sistema deve permitir salvar, alterar, excluir e listar os dados dos tutores (Código, Nome, Cidade, Telefone, CPF).
Propriedade	essencial

Identificador	RF 0002
Nome	Manter Cadastro de Animais
Descrição	O sistema deve permitir salvar, alterar, excluir e listar os dados dos animais (Código, Nome, Porte, Espécie, Raça, Sexo, Idade, Tutor vinculado).
Propriedade	essencial

Identificador	RF 0003
Nome	Manter Cadastro de Medicamentos
Descrição	O sistema deve permitir ao administrador salvar, alterar, excluir e listar medicamentos (Código, Nome, Indicação, Posologia, Preço, Qtd Estoque).
Propriedade	essencial

Identificador	RF 0004
Nome	Manter Cadastro de Veterinários
Descrição	O sistema deve permitir salvar, alterar, excluir e listar veterinários (Código, Nome, Celular, Turno, Salário, CRMV).
Propriedade	essencial

Identificador	RF 0005
Nome	Registrar Atendimento Veterinário
Descrição	O sistema deve permitir ao veterinário abrir um atendimento, selecionar o animal e o veterinário, definir o tipo (rotina/emergência) e digitar o resumo.
Propriedade	essencial

Identificador	RF 0006
Nome	Adicionar Itens (Medicamentos) ao Atendimento
Descrição	O sistema deve permitir adicionar medicamentos e suas respectivas quantidades durante a consulta.
Propriedade	essencial

Identificador	RF 0007
Nome	Calcular e Encerrar Atendimento
Descrição	O sistema deve somar o valor da consulta aos medicamentos utilizados e emitir o valor total a pagar no encerramento.
Propriedade	essencial

Identificador	RF 0008
Nome	Consulta de Tutores
Descrição	O sistema deve emitir uma listagem com os dados de todos os tutores cadastrados em ordem alfabética (Código, Nome, Cidade e Telefone).
Propriedade	essencial

Identificador	RF 0009
Nome	Consulta de Veterinários por Turno
Descrição	O sistema deve filtrar e listar todos os veterinários que trabalham no turno diurno.
Propriedade	essencial

Identificador	RF 0010
Nome	Consulta de Estoque de Medicamentos
Descrição	O sistema deve listar todos os medicamentos e exibir a sua situação atual no estoque.
Propriedade	essencial

Identificador	RF 0011
Nome	Consulta de Animais Atendidos por Veterinário
Descrição	O sistema deve mostrar todos os animais atendidos na clínica, associados ao veterinário responsável.
Propriedade	essencial

Identificador	RF 0012
Nome	Consulta de Número de Atendimentos por Ano
Descrição	O sistema deve exibir a quantidade de atendimentos efetuados de forma agrupada por ano.
Propriedade	essencial

Identificador	RF 0013
Nome	Menu Principal Integrador
Descrição	O sistema deve apresentar uma tela inicial com menus suspensos para acessar de forma organizada todos os módulos (Arquivo, Cadastros, Atendimento, Ajuda).
Propriedade	essencial

Identificador	RF 0014
Nome	Encerramento do Sistema
Descrição	O sistema deve permitir o encerramento seguro do aplicativo através do menu ou botão de fechar.
Propriedade	essencial

3.2. Requisitos Não Funcionais

Identificador	RNF 0001
Nome	Compatibilidade do Sistema
Descrição	O sistema deve funcionar corretamente no sistema operacional Windows.
Propriedade	Essencial

Identificador	RNF 0002
Nome	Desempenho
Descrição	O sistema deve consumir poucos recursos computacionais, não ultrapassando 50% da memória disponível das máquinas da clínica.
Propriedade	Essencial

Identificador	RNF 0003
Nome	Usabilidade
Descrição	O sistema deve possuir uma interface simples, limpa e intuitiva, facilitando sua utilização pelos funcionários.
Propriedade	Essencial

Identificador	RNF 0004
Nome	Segurança dos Dados
Descrição	O sistema deve garantir a realização de backup dos dados a cada 15 dias.
Propriedade	Essencial

Identificador	RNF 0005
Nome	Banco de Dados
Descrição	O sistema deve utilizar o MySQL para armazenamento das informações.
Propriedade	Essencial

Identificador	RNF 0006
Nome	Tecnologia de Desenvolvimento
Descrição	O sistema deve ser desenvolvido utilizando a linguagem de programação Java.
Propriedade	Essencial

Identificador	RNF 0007
Nome	Padronização das Classes
Descrição	Todas as classes do sistema devem possuir construtor padrão, construtor completo, métodos getters e setters e método toString.
Propriedade	Essencial

3.3. Regras de Negócio

Identificador	RN 0001
Nome	Controle de Espécies e Porte
Descrição	A clínica deve aceitar apenas animais das espécies cachorro, gato e pássaro, sendo permitidos somente animais de pequeno e médio porte.
Propriedade	Essencial

Identificador	RN 0002
Nome	Cadastro de Tutores
Descrição	Todo animal cadastrado no sistema deve possuir um tutor responsável vinculado, sendo obrigatório o uso de um CPF válido no cadastro.
Propriedade	Essencial

Identificador	RN 0003
Nome	Tipos de Atendimento
Descrição	O sistema deve permitir apenas atendimentos do tipo rotina e emergência.
Propriedade	Essencial

Identificador		RN 0004
Nome	Uso de Medicamentos	
Descrição	Durante o atendimento veterinário, o uso de medicamentos será opcional.	
Propriedade	Essencial	

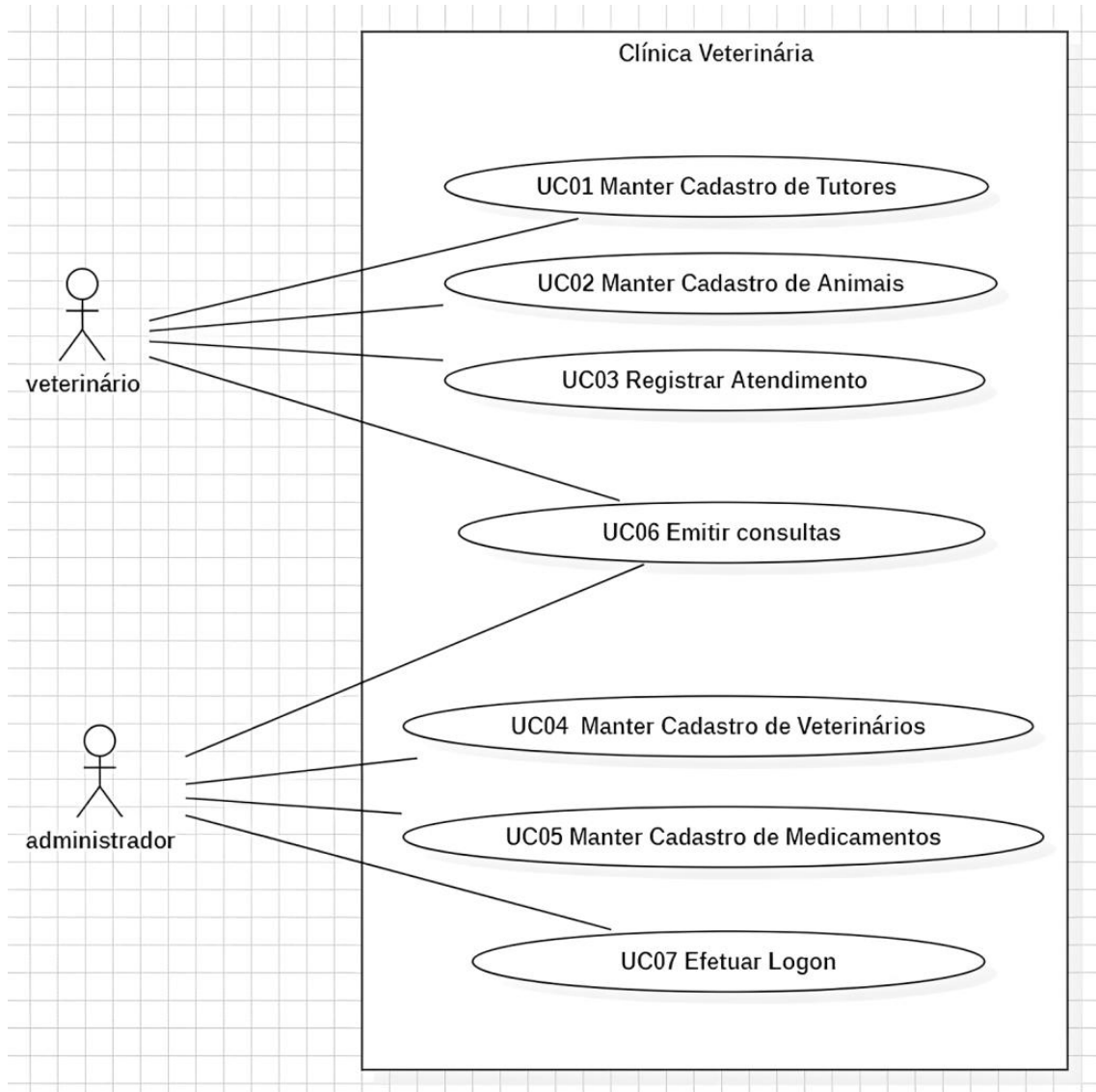
Identificador		RN 0005
Nome	Controle de Estoque	
Descrição	Um medicamento deve ser considerado com estoque baixo quando possuir quantidade inferior a 5 unidades.	
Propriedade	Essencial	

Identificador		RN 0006
Nome	Vínculo do Atendimento	
Descrição	Todo atendimento realizado deve estar associado a um animal e a um veterinário responsável.	
Propriedade	Essencial	

Identificador		RN 0007
Nome	Vínculo do Atendimento	
Descrição	Todo atendimento realizado deve estar associado a um animal e a um veterinário responsável.	
Propriedade	Essencial	

3.4. passo 2: diagrama de casos de uso

Atenção! as marcações UC que antecedem as numerações dos casos de uso se traduzem como “USE CASE”.



3.5. Descrição dos Casos de Uso

UC01 – Manter Cadastro de Tutores

Elemento	Descrição
Atores	Veterinário
Objetivo	Gerenciar o cadastro de tutores de animais na clínica.
Pré-condições	O usuário deve estar autenticado no sistema.
Fluxo Principal	1. O usuário acessa a tela de tutores. 2. O sistema exibe os campos: Código, Nome, Cidade, Telefone e CPF. 3. O usuário preenche os dados e clica em "Salvar". 4. O sistema valida os campos obrigatórios e a integridade do CPF. 5. O sistema grava os dados e confirma o sucesso da operação.

UC02 – Manter Cadastro de Animais

Elemento	Descrição
Atores	Veterinário
Objetivo	Gerenciar as informações dos animais atendidos.
Pré-condições	O tutor do animal já deve estar previamente cadastrado no sistema.
Fluxo Principal	1. O usuário acessa a tela de animais. 2. O sistema exibe os campos: Nome, Porte, Espécie, Raça, Sexo, Idade e seleção do Tutor. 3. O usuário insere as informações do animal e vincula ao seu respectivo tutor. 4. O usuário clica em "Salvar". 5. O sistema valida se a espécie (Cachorro, Gato, Pássaro) e o porte (Pequeno, Médio) estão dentro das regras permitidas. 6. O sistema grava os dados no banco de dados.

UC03 – Registrar Atendimento Veterinário

Elemento	Descrição
Atores	Veterinário
Objetivo	Registrar consultas, procedimentos e medicamentos consumidos no atendimento.
Pré-condições	O animal e o veterinário devem estar cadastrados no sistema.
Fluxo Principal	1. O usuário inicia um novo atendimento. 2. O sistema gera o número do código e insere a data atual. 3. O usuário vincula o Veterinário, o Animal, define o Tipo (Rotina/Emergência) e digita o resumo descritivo. 4. O usuário adiciona os medicamentos consumidos e suas quantidades. 5. O usuário encerra o atendimento. 6. O sistema calcula o valor total, atualiza o estoque de medicamentos e salva a ficha de atendimento.

UC04 – Manter Cadastro de Veterinários

Elemento	Descrição
Atores	Veterinário
Objetivo	Gerenciar o cadastro dos médicos veterinários que atuam na clínica.
Pré-condições	O usuário deve estar autenticado com privilégios de Administrador.
Fluxo Principal	1. O usuário acessa a tela de veterinários. 2. O sistema exibe os campos: Código, Nome, CRMV, Salário e o campo de seleção do Turno (Diurno/Noturno). 3. O usuário preenche as informações e clica em "Salvar". 4. O sistema valida se o turno escolhido é Diurno ou Noturno para garantir a integridade dos dados. 5. O sistema grava os dados no banco de dados e confirma o cadastro.

UC05 – Manter Cadastro de Medicamentos

Elemento	Descrição
Atores	Veterinário
Objetivo	Gerenciar as informações e a quantidade em estoque dos medicamentos da clínica.
Pré-condições	O usuário deve estar autenticado com privilégios de Administrador.
Fluxo Principal	1. O usuário acessa a tela de medicamentos. 2. O sistema exibe os campos: Código, Nome, Quantidade em Estoque e Valor Unitário. 3. O usuário insere os dados do medicamento e clica em "Salvar". 4. O sistema armazena as informações no banco de dados para controle de inventário.

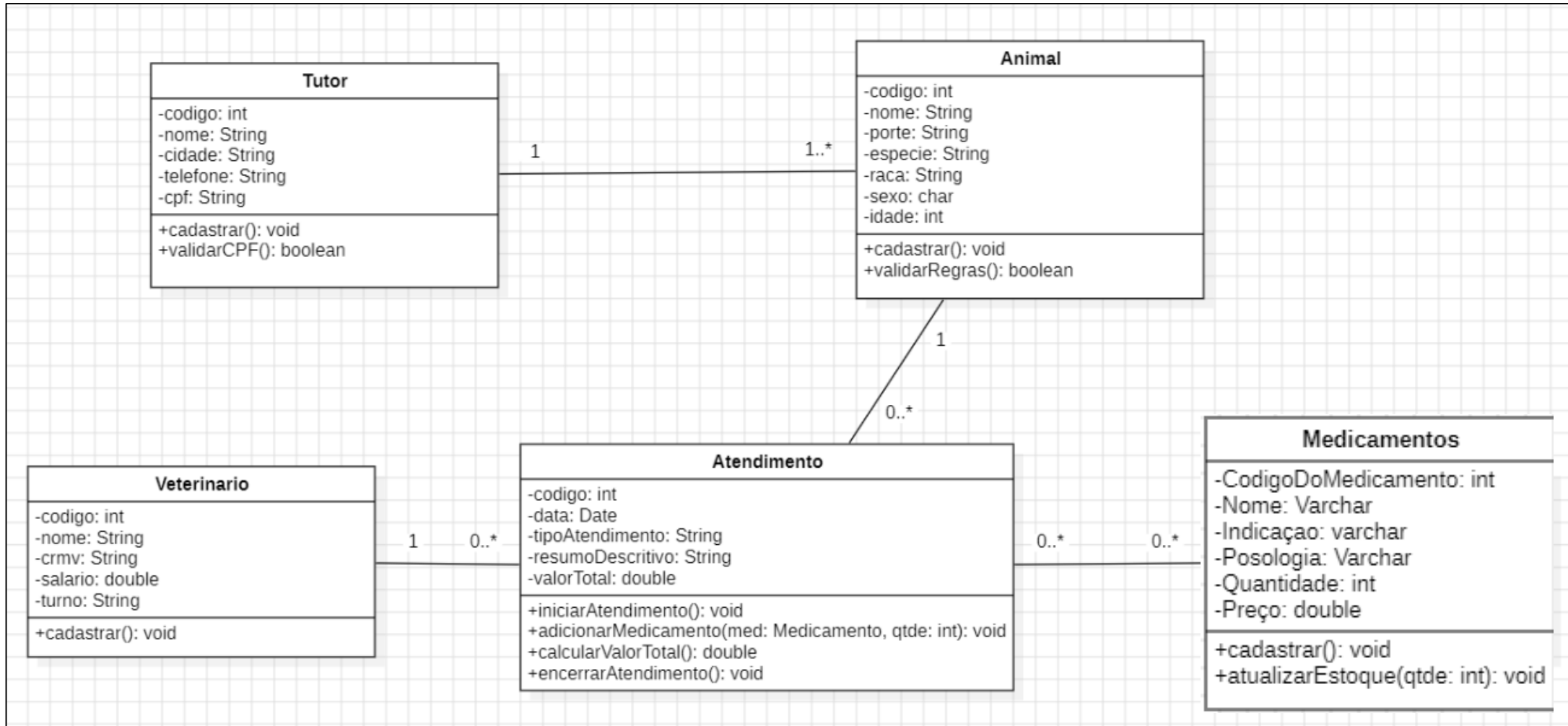
UC06 – Emitir Consultas Gerenciais

Elemento	Descrição
Atores	Veterinário
Objetivo	Disponibilizar relatórios e pesquisas rápidas extraídas do banco de dados.
Pré-condições	Existência de dados populados no sistema.
Fluxo Principal	1. O usuário acessa o menu de consultas. 2. O sistema exibe as 5 opções de filtros obrigatórios. 3. O usuário escolhe o relatório desejado e clica em "Buscar". 4. O sistema processa os dados do MySQL e exibe os resultados organizados em uma tabela na tela.

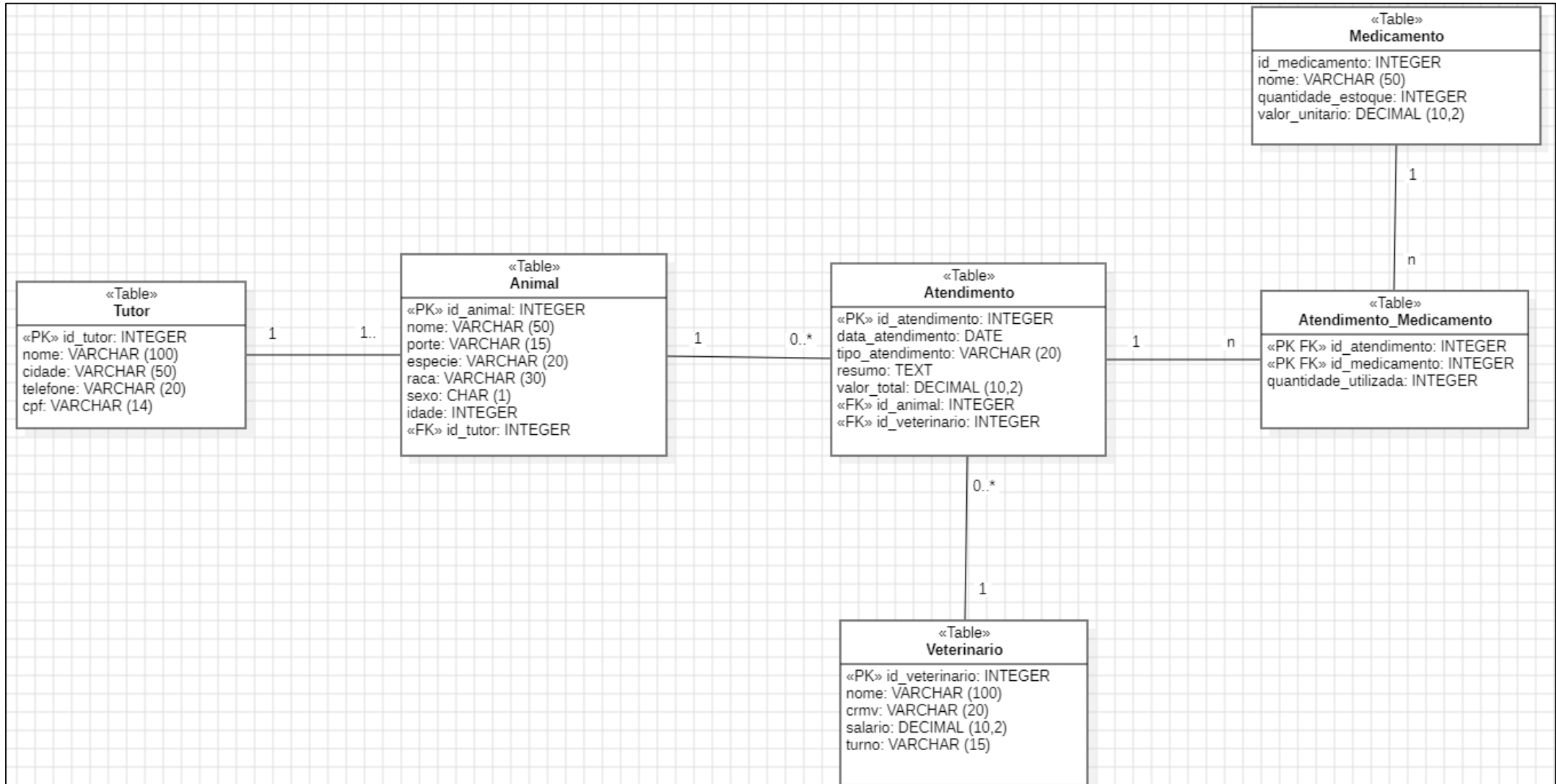
UC07 – Efetuar Logon no Sistema

Elemento	Descrição
Atores	Veterinário
Objetivo	Autenticar os usuários para liberar o acesso às funcionalidades do sistema.
Pré-condições	O usuário deve possuir um usuário e senha previamente cadastrados.
Fluxo Principal	1. O usuário digita suas credenciais de acesso na tela inicial. 2. O usuário clica em "Entrar". 3. O sistema valida os dados informados. 4. O sistema libera o acesso ao menu principal de acordo com o nível de permissão do ator.

3.6 passo 3: Diagrama de Classes



Etapa 2 – passo 4: Diagrama do Modelo Relacional.



Passo 5 – Definição do Banco de Dados e das Tabelas

COMANDO PARA CRIAÇÃO DO BANCO DE DADOS:

```
CREATE DATABASE CLINICA_VETERINARIA;
```

COMANDO PARA CRIAÇÃO DA TABELA TUTOR:

```
CREATE TABLE TUTOR (  
  ID_TUTOR INTEGER AUTO_INCREMENT PRIMARY KEY,  
  NOME VARCHAR (100) NOT NULL,  
  CIDADE VARCHAR (100) NOT NULL,  
  TELEFONE VARCHAR (20) NOT NULL,  
  CPF CHAR (14)  
);
```

TABELA TUTOR:

```
mysql> DESCRIBE TUTOR;
```

Field	Type	Null	Key	Default	Extra
ID_TUTOR	int	NO	PRI	NULL	auto_increment
NOME	varchar(100)	NO		NULL	
CIDADE	varchar(100)	NO		NULL	
TELEFONE	varchar(20)	NO		NULL	
CPF	char(14)	YES		NULL	

```
5 rows in set (0.00 sec)
```

COMANDO PARA CRIAÇÃO DA TABELA ANIMAL:

```
CREATE TABLE ANIMAL (  
  ID_ANIMAL INTEGER AUTO_INCREMENT PRIMARY KEY,  
  NOME VARCHAR (50) NOT NULL,  
  PORTE VARCHAR (15) NOT NULL,  
  ESPECIE VARCHAR (20) NOT NULL,  
  RACA VARCHAR (30) NOT NULL,  
  SEXO CHAR (1) NOT NULL,  
  IDADE INTEGER NOT NULL,  
  ID_TUTOR INT,  
  FOREIGN KEY (ID_TUTOR) REFERENCES TUTOR (ID_TUTOR)  
);
```

TABELA ANIMAL:

```
mysql> describe animal;
```

Field	Type	Null	Key	Default	Extra
ID_ANIMAL	int	NO	PRI	NULL	auto_increment
NOME	varchar(50)	NO		NULL	
PORTE	varchar(15)	NO		NULL	
ESPECIE	varchar(20)	NO		NULL	
RACA	varchar(30)	NO		NULL	
SEXO	char(1)	NO		NULL	
IDADE	int	NO		NULL	
ID_TUTOR	int	YES	MUL	NULL	

```
8 rows in set (0.01 sec)
```

COMANDO PARA CRIAÇÃO DA TABELA VETERINÁRIO:

```
CREATE TABLE VETERINARIO (  
  ID_VETERINARIO INTEGER AUTO_INCREMENT PRIMARY KEY,  
  NOME VARCHAR (100) NOT NULL,  
  CRMV VARCHAR (20) NOT NULL,  
  SALARIO DECIMAL (10,2) NOT NULL,  
  TURNO VARCHAR (15) NOT NULL  
);
```

TABELA VETERINARIO:

```
mysql> DESCRIBE VETERINARIO;
```

Field	Type	Null	Key	Default	Extra
ID_VETERINARIO	int	NO	PRI	NULL	auto_increment
NOME	varchar(100)	NO		NULL	
CRMV	varchar(20)	NO		NULL	
SALARIO	decimal(10,2)	NO		NULL	
TURNO	varchar(15)	NO		NULL	

```
5 rows in set (0.00 sec)
```

COMANDO PARA CRIAÇÃO DA TABELA ATENDIMENTO:

```
CREATE TABLE ATENDIMENTO (  
  ID_ATENDIMENTO INTEGER AUTO_INCREMENT PRIMARY KEY,  
  DATA_ATENDIMENTO DATE NOT NULL,  
  TIPO_ATENDIMENTO VARCHAR (20) NOT NULL,  
  VALOR_TOTAL DECIMAL (10,2) NOT NULL,  
  ID_ANIMAL INT,  
  ID_VETERINARIO INT,  
  FOREIGN KEY (ID_ANIMAL) REFERENCES ANIMAL (ID_ANIMAL),  
  FOREIGN KEY (ID_VETERINARIO) REFERENCES VETERINARIO (ID_VETERINARIO)  
);
```

TABELA ATENDIMENTO:

```
mysql> DESCRIBE ATENDIMENTO;
```

Field	Type	Null	Key	Default	Extra
ID_ATENDIMENTO	int	NO	PRI	NULL	auto_increment
DATA_ATENDIMENTO	date	NO		NULL	
TIPO_ATENDIMENTO	varchar(20)	NO		NULL	
VALOR_TOTAL	decimal(10,2)	NO		NULL	
ID_ANIMAL	int	YES	MUL	NULL	
ID_VETERINARIO	int	YES	MUL	NULL	

```
6 rows in set (0.01 sec)
```

COMANDO PARA CRIAÇÃO DA TABELA MEDICAMENTO:

```
CREATE TABLE MEDICAMENTO (  
  ID_MEDICAMENTO INTEGER AUTO_INCREMENT PRIMARY KEY,  
  NOME VARCHAR (50) NOT NULL,  
  POSOLOGIA TEXT NOT NULL,  
  INDICACAO VARCHAR (100) NOT NULL,  
  QUANTIDADE_ESTOQUE INTEGER NOT NULL,  
  VALOR_UNITARIO DECIMAL (10,2) NOT NULL  
);
```

TABELA MEDICAMENTO:

```
mysql> DESCRIBE MEDICAMENTO;
```

Field	Type	Null	Key	Default	Extra
ID_MEDICAMENTO	int	NO	PRI	NULL	auto_increment
NOME	varchar(50)	NO		NULL	
POSOLOGIA	text	NO		NULL	
INDICACAO	varchar(100)	NO		NULL	
QUANTIDADE_ESTOQUE	int	NO		NULL	
VALOR_UNITARIO	decimal(10,2)	NO		NULL	

```
6 rows in set (0.02 sec)
```

COMANDO PARA CRIAÇÃO DA TABELA ATENDIMENTO_MEDICAMENTO :

```
CREATE TABLE ATENDIMENTO_MEDICAMENTO(  
  ID_ATENDIMENTO INTEGER,  
  ID_MEDICAMENTO INTEGER,  
  QUANRIDADE_UTILIZADA INTEGER NOT NULL,  
  PRIMARY KEY (ID_ATENDIMENTO, ID_MEDICAMENTO),  
  FOREIGN KEY (ID_ATENDIMENTO) REFERENCES ATENDIMENTO (ID_ATENDIMENTO),  
  FOREIGN KEY (ID_MEDICAMENTO) REFERENCES MEDICAMENTO(ID_MEDICAMENTO)  
);
```

TABELA ATENDIMENTO_MEDICAMENTO:

```
mysql> DESCRIBE ATENDIMENTO_MEDICAMENTO;  
+-----+-----+-----+-----+-----+-----+  
| Field                | Type | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| ID_ATENDIMENTO       | int  | NO   | PRI | NULL    |       |  
| ID_MEDICAMENTO       | int  | NO   | PRI | NULL    |       |  
| QUANRIDADE_UTILIZADA | int  | NO   |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
3 rows in set (0.01 sec)
```


Passo 6 – Definição das Consultas de Dados

- Consulta 1: **Consulta de Tutores** Esta consulta deve mostrar uma listagem com os dados de todos os tutores cadastrados em ordem alfabética.

A consulta precisa conter o código do tutor, seu nome, cidade e telefone.

```
SELECT ID_TUTOR,NOME,CIDADE,TELEFONE  
FROM TUTOR  
ORDER BY NOME ASC;
```

- Consulta 2: **Consulta de Veterinários por Turno** Esta consulta deve mostrar uma lista com todos os veterinários que trabalham no turno diurno.

A consulta precisa conter o código do veterinário, o nome do veterinário, o celular e o turno.

```
SELECT ID_VETERINARIO,NOME,CELULAR,TURNIO  
FROM VETERINARIO  
WHERE turno = 'DIURNO';
```

- Consulta 3: **Consulta de Estoque de Medicamentos** Esta consulta deve mostrar a listagem de todos os medicamentos, exibindo a sua situação no estoque.

A consulta precisa conter o código do medicamento, o seu nome, preço e quantidade em estoque.

```
SELECT ID_MEDICAMENTO,NOME,PRECO,QUANTIDADE_ESTOQUE  
FROM MEDICAMENTO;
```

- Consulta 4: **Consulta de Animais Atendidos por Veterinário** Esta consulta deve mostrar todos os animais que foram atendidos na clínica, juntamente com o veterinário responsável pelo atendimento.

A consulta precisa conter o código do atendimento, a data do atendimento, o nome do animal e o nome do veterinário.

```
SELECT ID_ATENDIMENTO, DATA_ATENDIMENTO, NOME AS ANIMAL,  
NOME AS VETERINARIO  
FROM ATENDIMENTO INNER JOIN ANIMAL  
USING(ID_ANIMAL)  
INNER JOIN VETERINARIO  
USING(ID_VETERINARIO);
```

- Consulta 5: **Consulta de Número de Atendimentos por Ano** Esta consulta deve mostrar o número de atendimentos efetuados, agrupando os atendimentos por ano.

A consulta precisa conter apenas o ano e o número de atendimentos correspondente.

```
SELECT YEAR(DATA_ATENDIMENTO), COUNT(*)  
FROM ATENDIMENTO  
GROUP BY YEAR(DATA_ATENDIMENTO);
```

Etapa 3 – Implementação do Software

Passo 7 – Criação das Classes

```
public class Tutor {  
  
    // 1. Atributos privados correspondentes às colunas da tabela TUTOR  
    private int id_tutor;  
    private String nome;  
    private String cidade;  
    private String telefone;  
    private String cpf;  
  
    // 2. Construtor Padrão (Sem parâmetros) - Obrigatório pelo RNF 0007  
    public Tutor() {  
    }  
  
    // 3. Construtor Completo (Com todos os parâmetros) - Obrigatório pelo RNF 0007  
    public Tutor(int id_tutor, String nome, String cidade, String telefone, String cpf) {  
        this.id_tutor = id_tutor;  
        this.nome = nome;  
        this.cidade = cidade;  
        this.telefone = telefone;  
        this.cpf = cpf;  
    }  
}
```

```
// 4. Métodos Getters e Setters para todos os atributos
```

```
public int getId_tutor() {  
    return id_tutor;  
}
```

```
public void setId_tutor(int id_tutor) {  
    this.id_tutor = id_tutor;  
}
```

```
public String getNome() {  
    return nome;  
}
```

```
public void setNome(String nome) {  
    this.nome = nome;  
}
```

```
public String getCidade() {  
    return cidade;  
}
```

```
public void setCidade(String cidade) {  
    this.cidade = cidade;  
}
```

```
public String getTelefone() {  
    return telefone;  
}
```

```
public void setTelefone(String telefone) {  
    this.telefone = telefone;  
}  
public String getCpf() {  
    return cpf;  
}  
  
public void setCpf(String cpf) {  
    this.cpf = cpf;  
}  
  
// 5. Método toString() - Obrigatório pelo RNF 0007  
@Override  
public String toString() {  
    return "Tutor{" + "id_tutor=" + id_tutor + ", nome=" + nome + ", cidade=" + cidade + ",  
telefone=" + telefone + ", cpf=" + cpf + '}';  
}
```

Passo 8 – Definição das Interfaces

Formulário de Cadastro de Tutores

Source Design History

To change layout manager of a container use Set Layout submenu from its context menu.

Código:

Nome:

Cidade: Telefone:

CPF:

Lista de Tutores

Código	Nome	Cidade	Telefone	CPF

Salvar Alterar Excluir Listar Fechar

Formulário de Cadastro de Animais

cadastro de animais

Codigo:

Nome Porte:

Espécie: Raça:

Sexo: Idade:

Tutor:

Title 1	Title 2	Title 3	Title 4

Formulário de Cadastro de Medicamentos

cadastro de animais

Codigo:

Nome Porte:

Espécie: Raça:

Sexo: Idade:

Tutor:

Title 1	Title 2	Title 3	Title 4

Formulário de Cadastro de Veterinários

cadastro de veterinarios

codigo

nome

celular salario

turno crmv

lista de veterinarios

codigo	nome	celular	salario	turno	crm

salvar alterar excluir listar listar

Formulário de Cadastro de Atendimentos

codigo

data tipo de atendimento

resumo

animal atendimento

veterinario

valor consulta

Title 1	Title 2	Title 3	Title 4

total a pagar

novo medicamento adicionar item remover item encerrar atendimento fechar

Formulário de cadastro medicamento

Formulário de cadastro medicamento

codigo

nome

indicação posologia

preço quantidade em estoque

lista de medicamentos

Title 1	Title 2	Title 3	Title 4

salvar alterar excluir listar fechar

Formulário Principal do Sistema

Arquivo Cadastro Atendimento Ajuda

☐ Tutores

☐ Animais

☐ Medicamentos

☐ Veterinários

código Botão ajuda :

```
private void jMenuItem7ActionPerformed(java.awt.event.ActionEvent evt) {  
  
    String info = "--- CLINICA VETERINÁRIA ---\n\n"  
        + "Versão: 1.0\n"  
        + "Desenvolvido por: DIEGO ALMEIDA DA SILVA-\n" +  
        "JEFERSON HENRIQUE DE PAULA FERNANDES-\n" +  
        "GABRIEL DE ARAUJO CARRIJO SOARES DA SILVEIRA\n"  
        + "Curso: Técnico em Informática - FAETEC\n\n"  
        + "Sistema de gerenciamento e cadastro de tutores\n"  
        + "integrado com banco de dados MySQL.\n\n"  
        + "Todos os direitos reservados © 2026.";  
  
    javax.swing.JOptionPane.showMessageDialog(null, info, "Informações do Aplicativo",  
        javax.swing.JOptionPane.INFORMATION_MESSAGE);  
}
```

código Tutor cadastro:

botão salvar:

```
private void btnSalvarActionPerformed(java.awt.event.ActionEvent evt) {  
  
    Tutor t = new Tutor();  
  
    t.setNome(txtNome.getText());  
    t.setCidade(txtCidade.getText());  
    t.setTelefone(txtTelefone.getText());  
    t.setCpf(txtCpf.getText());  
  
    t.salvar();  
  
    txtCodigo.setText("");  
    txtNome.setText("");  
    txtCidade.setText("");  
    txtTelefone.setText("");  
    txtCpf.setText("");  
  
}
```

Botão Alterar:

```
private void btnAlterarActionPerformed(java.awt.event.ActionEvent evt) {  
  
    Tutor t = new Tutor();  
  
    t.setId_tutor(Integer.parseInt(txtCodigo.getText()));  
    t.setNome(txtNome.getText());  
    t.setCidade(txtCidade.getText());  
    t.setTelefone(txtTelefone.getText());  
    t.setCpf(txtCpf.getText());  
  
    t.alterar();  
  
    txtCodigo.setText("");  
    txtNome.setText("");  
    txtCidade.setText("");  
    txtTelefone.setText("");  
    txtCpf.setText("");  
  
}
```

botão excluir

```
private void btnExcluirActionPerformed(java.awt.event.ActionEvent evt) {  
  
    Tutor t = new Tutor();  
  
    t.setId_tutor(Integer.parseInt(txtCodigo.getText()));  
  
    t.excluir();  
  
    txtCodigo.setText("");  
    txtNome.setText("");  
    txtCidade.setText("");  
    txtTelefone.setText("");  
    txtCpf.setText("");  
  
}
```

botão listar:

```
private void txtListarActionPerformed(java.awt.event.ActionEvent evt) {  
  
    Tutor t = new Tutor();  
    java.sql.ResultSet rs = t.listar();  
  
    // Cria uma string para acumular o texto de todos os tutores  
    String listaCompleta = "--- LISTA DE TUTORES CADASTRADOS ---\n\n";  
  
    try {  
        boolean temDados = false;  
        while (rs != null && rs.next()) {  
            temDados = true;  
  
            // Puxa os dados linha por linha e joga na string  
            listaCompleta += "ID: " + rs.getInt("ID_TUTOR") + "\n"  
                + "Nome: " + rs.getString("NOME") + "\n"  
                + "Cidade: " + rs.getString("CIDADE") + "\n"  
                + "Telefone: " + rs.getString("TELEFONE") + "\n"  
                + "CPF: " + rs.getString("CPF") + "\n"  
                + "-----\n";  
        }  
  
        if (!temDados) {  
            listaCompleta = "Nenhum tutor encontrado no banco de dados.";  
        }  
  
        javax.swing.JOptionPane.showMessageDialog(null, listaCompleta);  
    } catch (Exception erro) {  
        javax.swing.JOptionPane.showMessageDialog(null, "Erro ao processar lista: " +  
            erro.getMessage());  
    }  
}
```

Conexão com o banco de dados

Método salvar:

```
// metodo salvar

public void salvar() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");

        Connection conexao =
        DriverManager.getConnection("jdbc:mysql://localhost:3306/CLINICA_VETERINARIA",
        "root", "1234");

        Statement comando = conexao.createStatement();

        String sql = "INSERT INTO TUTOR (NOME, CIDADE, TELEFONE, CPF) VALUES ("
            + this.nome + ", "
            + this.cidade + ", "
            + this.telefone + ", "
            + this.cpf + ")";

        comando.executeUpdate(sql);

        JOptionPane.showMessageDialog(null, "Tutor cadastrado com sucesso!");

        comando.close();
        conexao.close();
    } catch (Exception erro) {
        JOptionPane.showMessageDialog(null, "Erro ao salvar tutor no banco: " +
        erro.getMessage());
    }
}
```

Método alterar:

```
// metodo alterar

public void alterar() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");

        Connection conexao =
        DriverManager.getConnection("jdbc:mysql://localhost:3306/CLINICA_VETERINARIA",
        "root", "1234");

        Statement comando = conexao.createStatement();

        String sql = "UPDATE TUTOR SET NOME = " + this.nome
            + ", CIDADE = " + this.cidade
            + ", TELEFONE = " + this.telefone
            + ", CPF = " + this.cpf
            + " WHERE ID_TUTOR = " + this.id_tutor;

        comando.executeUpdate(sql);

        JOptionPane.showMessageDialog(null, "Tutor alterado com sucesso!");

        comando.close();
        conexao.close();
    } catch (Exception erro) {
        JOptionPane.showMessageDialog(null, "Erro ao alterar tutor no banco: " +
        erro.getMessage());
    }
}
```


Método excluir:

```
//metodo excluir

public void excluir() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");

        Connection conexao =
DriverManager.getConnection("jdbc:mysql://localhost:3306/CLINICA_VETERINARIA",
"root", "1234");

        Statement comando = conexao.createStatement();

        String sql = "DELETE FROM TUTOR WHERE ID_TUTOR = " + this.id_tutor;

        comando.executeUpdate(sql);

        JOptionPane.showMessageDialog(null, "Tutor excluído com sucesso!");

        comando.close();
        conexao.close();
    } catch (Exception erro) {
        JOptionPane.showMessageDialog(null, "Erro ao excluir tutor no banco: " +
erro.getMessage());
    }
}
```

Método listar:

```
// metodo listar

public java.sql.ResultSet listar() {
    java.sql.ResultSet resultado = null;
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        java.sql.Connection conexao =
        java.sql.DriverManager.getConnection("jdbc:mysql://localhost:3306/CLINICA_VETERI
        NARIA", "root", "1234");

        java.sql.Statement comando = conexao.createStatement();

        String sql = "SELECT * FROM TUTOR ORDER BY NOME";
        resultado = comando.executeQuery(sql);

    } catch (Exception erro) {
        javax.swing.JOptionPane.showMessageDialog(null, "Erro ao listar tutores: " +
        erro.getMessage());
    }
    return resultado;
}
}
```